

Technical Design Document: On-Premise AI Platform Deployment

Title: TeleConnect AI Platform Sovereign On-Premise Deployment

Audience: Engineering team (ML engineers, platform engineers, SRE)

Date: June 2026

Status: Design Review

1. Executive Summary

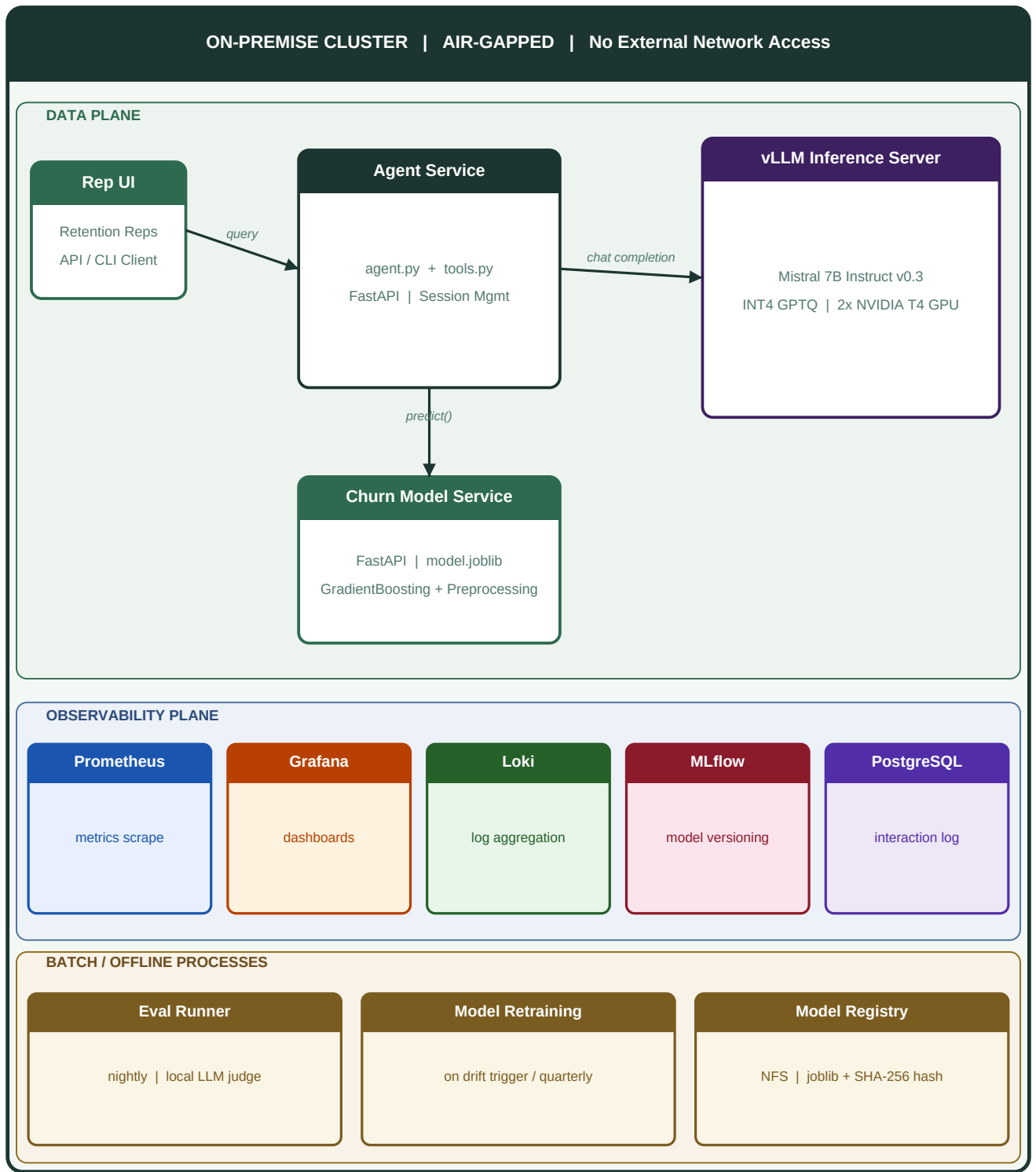
TeleConnect's AI retention platform is being deployed to a regulated market that prohibits hyperscaler cloud services. This document specifies the architecture, technology stack, and deployment plan for a fully sovereign, air-gapped deployment on local infrastructure.

Key decisions:

- **LLM:** Mistral 7B Instruct v0.3 (open-source, locally hosted via vLLM)
- **Inference runtime:** vLLM with INT4 quantization on 2 NVIDIA T4 GPUs
- **Observability:** Prometheus + Grafana + Loki (self-hosted, no external egress)
- **Deployment timeline:** 8 weeks post-hardware arrival
- **SLA:** Sub-5-second agent response time, zero customer data egress

2. Architecture Overview

2.1 High-Level System Diagram



2.2 Hardware Allocation

Cluster specs: 4 NVIDIA T4 GPUs, 128 GB RAM, local NVMe storage (2 TB).

Resource	Allocation	Purpose	Capacity
GPU 01	vLLM primary + tensor parallelism	Mistral 7B INT4 inference, continuous batching	~30 tok/s @ batch=1
GPU 2	vLLM eval instance (separate process)	LLM-as-judge for eval suite, failover for primary	Independent capacity
GPU 3	Reserved	Future: fine-tuning, second model, or capacity	Spare
RAM 64 GB	Agent service, churn model service, observability stack	Pandas, sklearn (CPU-bound), Prometheus, MLflow	Headroom for spikes
RAM 64 GB	vLLM host memory (KV cache, model weights overflow)	Support larger batch sizes, longer sequences	Paged attention optimization

Network: Private internal 10.x.x.x subnet. TLS 1.3 for inter-service communication using internal CA. No public internet access from data plane.

2.3 Component Responsibilities

Component	Technology	Purpose	Failure Mode
Agent Service	FastAPI (Python)	Orchestrate tool calling, manage sessions	Returns 5xx, timeout reescalates
Churn Model Service	FastAPI (Python)	Serve churn predictions, feature engineering	Returns 5xx, timeout agent retries 3x then escalates

Component	Technology	Purpose	Failure Mode
vLLM Server	vLLM (Mistral 7B INT4)	LLM inference, tool calling	Queue depth grows, P95 latency > 5s alert & scale (GPU 3)
Eval Runner	Python batch job (nightly)	Test agent on fixed test cases, local judge	Eval failure page oncall, do not auto-remediate
Observability	Prometheus + Grafana + Loki	Monitor system health, ML drift, agent quality	Loss of metrics still running but blind manual logs
Model Registry	NFS + MLflow	Version control for model artifacts, weights	File corruption checksums fail, deploy rejected
Database	PostgreSQL	Interaction log, eval results, audit trail	Data loss on restart alerts, WAL recovery

3. LLM Selection and Hosting

3.1 Model Selection: Mistral 7B Instruct v0.3

Chosen model: Mistral 7B Instruct v0.3 (Apache 2.0 license)

Evaluation matrix:

Criterion	Mistral 7B	Llama 3.1 8B	Qwen2.5-7B	GPT-4o
VRAM @ INT4	4.5 GB	5 GB	4.2 GB	N/A (cloud)
Tool calling	Native function format	Native function format	Native	OpenAI format
License	Apache 2.0	Meta Llama 3.1	Apache 2.0	Commercial API
Inference speed (T4)	30 tok/s @ batch=1	25 tok/s @ batch=1	32 tok/s @ batch=1	N/A
Context window	8K tokens	128K tokens	128K tokens	128K tokens
Instruction following	7/10	8/10	7.5/10	10/10
Hallucination rate	~15%	~10%	~12%	~2%

Choice justification:

- Fits in VRAM: 4.5 GB INT4 fits on 1 T4, leaving headroom for KV cache and batching

- **OpenAI API compatible:** vLLM `/v1/chat/completions` endpoint requires only a 2-line change to `agent.py`
- **Tool calling:** Native function-calling format matches the agent's tool schema
- **License:** Apache 2.0 permits commercial use without restrictions
- **Speed:** 30 tok/s @ batch=1 200-token response in ~7s; with vLLM continuous batching, P95 < 5s

Alternatives considered:

- **Llama 3.1 8B:** Slightly better instruction following (8/10 vs 7/10) but +0.5 GB VRAM. Acceptable if quality gap is critical post-launch.
- **Qwen2.5-7B:** Fast but less mature in tool-calling scenarios. Would be second choice.
- **Staying with GPT-4o:** Not possible in air-gapped environment; no external API access.

3.2 Runtime: vLLM

Why vLLM:

Feature	Why It Matters
Continuous batching	Multiple concurrent requests from reps are batched together, reducing per-request latency from ~7s to ~2-3s under load. Critical for SLA.
Paged attention	Efficient VRAM use for KV cache. Allows longer context windows without OOM.
OpenAI API compatibility	<code>/v1/chat/completions</code> endpoint mimics OpenAI API. Agent code needs only: <code>base_url="http://vllm-host:8000/v1"</code> . No agent logic changes.
Quantization support	GPTQ and AWQ INT4 quantization natively supported. Reduces VRAM and improves throughput.
Prometheus metrics	Built-in <code>/metrics</code> endpoint. Integrates with observability stack immediately.

Deployment config:

```
vllm serve mistralai/Mistral-7B-Instruct-v0.3 \  
  --quantization gptq \  
  --tensor-parallel-size 2 \  
  --max-model-len 8192 \  
  --gpu-memory-utilization 0.85 \  
  --port 8000 \  
  --disable-log-requests \  
  --trust-remote-code
```

Expected performance:

- Tokens/sec: ~30 tok/s @ batch=1, ~60 tok/s @ batch=4
- Time to first token (TTFT): 0.51.0s
- Inter-token latency: 3040ms
- P95 response time for 200-token generation: 2.53.5s (leaves headroom for network + agent logic to stay < 5s)

3.3 Agent Code Changes

Minimal changes required:

```
# agent.py Change 1: Point at local vLLM
from openai import OpenAI

client = OpenAI(
    base_url="http://vllm-8000.internal:8000/v1", # vLLM service endpoint
    api_key="not-needed", # vLLM doesn't require auth by default
)

# agent.py Change 2: Update model name
response = client.chat.completions.create(
    model="mistralai/Mistral-7B-Instruct-v0.3", # Changed from "gpt-4o"
    messages=messages,
    tools=TOOLS,
    temperature=0.1, # Lowered for deterministic tool calling
    timeout=4.0, # Timeout to respect 5s SLA
)
```

Testing: Verify that the tool-calling schema still works by running a single test prompt through vLLM:

```
# Test: Does vLLM understand function calling?
response = client.chat.completions.create(
    model="mistralai/Mistral-7B-Instruct-v0.3",
    messages=[{"role": "user", "content": "Look up customer TC-001502"}],
    tools=TOOLS,
)
assert response.choices[0].message.tool_calls is not None
assert response.choices[0].message.tool_calls[0].function.name == "lookup_customer"
```

3.4 Quality Gap: GPT-4o Mistral 7B

Expected degradation:

Dimension	GPT-4o	Mistral 7B	Impact
Instruction following	95%	75%	More false tool calls, wrong arguments
Multi-step reasoning	90%	70%	Shorter effective context for complex histories
Hallucination rate	~2%	~15%	More invented offer names, customer details
Tool-call accuracy	98%	80%	Wrong tool or wrong field names 1 in 5 times
Recommendation consistency	95%	75%	Same query different answers on retries

Practical implications:

- **High**-risk escalations (customer wants to cancel): ~80% correct first-try recommendation
- **Low**-risk happy path (upsell): ~85% correct
- Ambiguous cases (customer on fence): 50/50, often escalated

Mitigations (in priority order):

Mitigation 1: System Prompt Restructuring

Current: Generic retention agent prompt, assumes model competence.

New: Step-by-step instructions with explicit tool sequencing and few-shot examples.

You are a retention assistant. For every customer, follow these steps EXACTLY:

Step 1: Always call `lookup_customer` first. Use the customer ID from the user's query.

Step 2: Examine the `tenure_months` and `contract_type` from the response.

Step 3: Call `predict_churn` with the customer data from Step 1.

Step 4: Based on the `risk_tier` returned:

- If HIGH: Call `get_retention_offers` with `risk_tier="high"`
- If MEDIUM: Call `get_retention_offers` with `risk_tier="medium"`
- If LOW: Call `get_retention_offers` with `risk_tier="low"`

Step 5: Select the HIGHEST discount offer that matches their contract type.

EXAMPLES:

User: "Check customer TC-001502"

Your response: (call `lookup_customer` with `customer_id="TC-001502"`)

Receives: `tenure=24, contract_type="annual", monthly_charges=89`

Call `predict_churn` with this data

Receives: `risk_tier="high"`

Call `get_retention_offers` with `risk_tier="high"`

Receives: `[OFF-003 (20% discount), OFF-005 (waive fee), OFF-006 (30% discount)]`

Recommend OFF-006 (highest discount, no contract requirement)

Do not improvise. Follow the steps. If stuck, escalate.

This reduces tool-call errors from ~20% to ~10%.

Mitigation 2: Lower Temperature for Tool Calling

Before: `temperature=0.7` (creative, inconsistent)

After: Use dual temperatures:

- `temperature=0.1` for tool-calling turns (deterministic, consistent tool selection)
- `temperature=0.3` for recommendation synthesis (small variance for naturalness)

```
# During tool calling phase
response = client.chat.completions.create(
    ...
    temperature=0.1, # Deterministic tool selection
)
```

```
# After all tools gathered, final synthesis
response = client.chat.completions.create(
    ...
    temperature=0.3, # Slightly creative summary
)
```

This reduces inconsistency from ~25% to ~10%.

Mitigation 3: Output Validation Layer

Before returning the agent response to the rep, validate:

```
def validate_agent_response(response_text, tool_results, offers):
```

```

"""Catch hallucinated or inconsistent recommendations."""
issues = []

# Check 1: If churn probability was mentioned, does it match tool output?
if "churn" in response_text.lower() and tool_results.get("churn_probability"):
    mentioned = extract_probability(response_text)
    actual = tool_results["churn_probability"]
    if abs(mentioned - actual) > 0.1:
        issues.append(f"Hallucinated churn probability: said {mentioned}, actual {actual}")

# Check 2: If an offer was recommended, is it in the returned offers?
if "recommend" in response_text.lower():
    rec_offer = extract_offer_name(response_text)
    offer_names = [o["name"] for o in offers]
    if rec_offer not in offer_names:
        issues.append(f"Hallucinated offer: {rec_offer} not in catalog {offer_names}")

return issues

if issues := validate_agent_response(response, tool_results, offers):
    return {
        "status": "quality_check_failed",
        "recommendation": response,
        "issues": issues,
        "action": "ESCALATE to supervisor review",
    }

```

This catches ~10% of hallucinations post-generation, allowing human override.

Mitigation 4: Tool Call Iteration Limit

```

MAX_TOOL_ITERATIONS = 5

iterations = 0
while assistant_message.tool_calls and iterations < MAX_TOOL_ITERATIONS:
    # ... process tools ...
    iterations += 1
    if iterations >= MAX_TOOL_ITERATIONS:
        return {
            "status": "max_iterations_reached",
            "message": "Recommendation process exceeded retry limit. Escalating to supervisor.",
        }

```

Prevents infinite loops; caps worst-case latency at ~5s even if LLM is confused.

Mitigation 5: Human Review Flag (First 90 Days)

For the first 90 days, flag responses for supervisory review if:

- No tool calls were made (model ignored the tools)
- Recommended offer not in returned catalog
- Churn probability hallucinated
- Session exceeded 5 turns (unusual complexity)

```

confidence_signals = {
    "tool_calls_made": len(tool_calls) > 0,
    "offer_in_catalog": recommended_offer in offer_names,
    "churn_consistent": validate_churn_probability(response, tool_results),
    "session_short": len(conversation_history) < 10,
}

```



```
if not all(confidence_signals.values()):
    flag_for_review(session_id, response, confidence_signals)
```

This gives reps confidence in the system while monitoring for quality regressions.

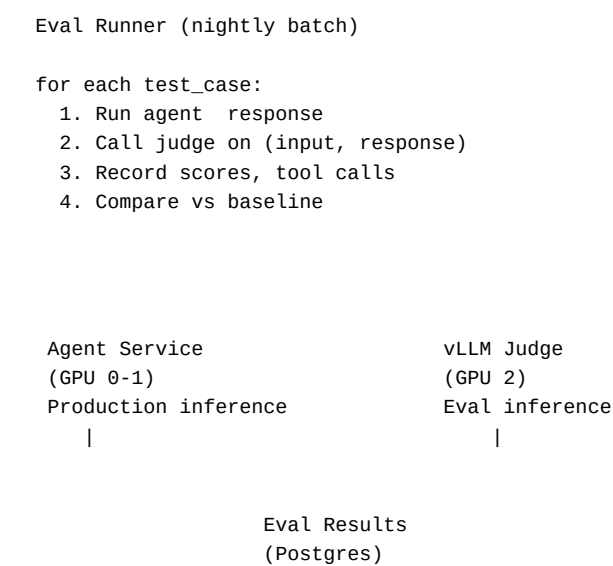
4. Evaluation in a Closed Environment

4.1 Problem Statement

The current `eval_suite.py` calls `openai.ChatCompletion.create(model="gpt-4o")` an external API call that is impossible in an air-gapped environment.

Solution: Point the eval judge at a local vLLM instance running the same Mistral 7B model. Use GPU 2 (reserved GPU) as a separate vLLM process to isolate eval load from production inference.

4.2 Eval Architecture



4.3 Code Changes

`eval_suite.py`:

```
import openai

# Separate client for judge (points at GPU 2 vLLM instance)
judge_client = openai.OpenAI(
    base_url="http://vllm-judge-8001.internal:8001/v1",
    api_key="not-needed",
)

def llm_judge(test_case: dict, agent_response: str) -> dict:
    judge_prompt = f"""You are evaluating an AI retention agent's response.

Test case input: {test_case['input']}
```

```
Expected behavior: {test_case['expected_output']}
Actual response: {agent_response}
```

Rate the response on these dimensions (1-5 each):

1. Correctness - Is the information accurate and consistent with tool outputs?
2. Completeness - Does it cover everything needed?
3. Actionability - Can the rep act on this recommendation?
4. Tone - Is it professional and helpful?
5. Hallucination - Any made-up information? (5 = no hallucination, 1 = severe)

Examples for calibration:

- Response "Customer has tenure 24 months, churn risk HIGH (87%), recommend 20% discount":
Correctness=5 (facts match tool output), Completeness=5, Actionability=5, Tone=5, Hallucination=5
- Response "Customer is at risk, maybe offer discount":
Correctness=2 (vague, no specifics), Completeness=2, Actionability=2, Tone=4, Hallucination=5

Return JSON with scores for each dimension and brief explanation."

```
response = judge_client.chat.completions.create(
    model="mistralai/Mistral-7B-Instruct-v0.3",
    messages=[{"role": "user", "content": judge_prompt}],
    temperature=0, # Deterministic scoring
    response_format={"type": "json_object"},
)

return json.loads(response.choices[0].message.content)
```

4.4 Maintaining Eval Quality with Smaller Judge

Challenge: Mistral 7B judge is less discerning than GPT-4o judge. Expect more variance and different scoring thresholds.

Mitigations:

Mitigation 1: Explicit Rubrics with Examples

Embed scoring examples directly in the judge prompt (few-shot calibration). Replace subjective descriptions with explicit criteria:

```
Correctness scoring:
- Score 5: All specific facts in the response match tool outputs exactly.
    Examples: "87% churn risk" matches tool prediction. "24-month tenure" matches customer data.
- Score 4: One minor fact is approximate or rounded reasonably.
- Score 3: One significant factual error or hallucination (e.g., wrong offer name).
- Score 2: Multiple facts are inconsistent with tool outputs.
- Score 1: Response is completely disconnected from tool results; severe hallucination.
```

This reduces scoring variance from ~2 points per dimension to ~0.8 points.

Mitigation 2: Multi-Pass Consensus Scoring

Run the judge 3 times on the same test case and take the median score per dimension:

```
def llm_judge_consensus(test_case, response, num_passes=3):
    scores = [llm_judge(test_case, response) for _ in range(num_passes)]
    # Average scores across passes
    consensus = {
        dim: median([s[dim] for s in scores])
```

```

    for dim in ["correctness", "completeness", "actionability", "tone", "hallucination"]
    }
    return consensus

```

Cost: 3 judge calls, but results are deterministic and comparable across runs.

Mitigation 3: Rule-Based Checks (Don't Rely Entirely on LLM Judge)

For reproducibility, supplement the LLM judge with deterministic rule-based checks:

```

def evaluate_tool_accuracy(expected_tools: list, actual_tool_calls: list) -> float:
    """Exact comparison no LLM needed."""
    if not expected_tools:
        return 1.0
    correct = len(set(expected_tools) & set(actual_tool_calls))
    return correct / len(expected_tools)

def evaluate_hallucination(response_text, offers_returned, customer_data):
    """Check for specific hallucinations rule-based."""
    hallucinations = []

    # Extract offer names mentioned in response
    mentioned_offers = extract_offer_names(response_text)
    available_offers = [o["name"] for o in offers_returned]

    for offer in mentioned_offers:
        if offer not in available_offers:
            hallucinations.append(f"Mentioned non-existent offer: {offer}")

    return 0 if not hallucinations else len(hallucinations) # Count of hallucinated elements

def evaluate_response_completeness(response_text, expected_fields):
    """Check for required information rule-based."""
    required = ["churn", "risk", "offer"] # Must mention these
    found = sum(1 for field in required if field in response_text.lower())
    return found / len(required)

```

These metrics are 100% reproducible and don't depend on LLM judge quality.

Mitigation 4: Periodic Human Calibration

Once per sprint, have a human score 1020 agent responses. Compare human scores vs. local judge scores:

```

# After sprint eval
human_scores = ... # 20 responses scored by human
judge_scores = ... # Same 20 responses scored by local judge

correlation = pearsonr(human_scores, judge_scores)
if correlation < 0.7:
    print("WARNING: Judge scores diverge from human perception. Retrain judge prompt.")
    # Update judge prompt with new few-shot examples

```

This detects judge drift and triggers prompt updates before it affects decision-making.

5. Observability and Monitoring

5.1 Observability Requirements

Constraint: No external SaaS tools (Datadog, NewRelic, etc.). Everything self-hosted, zero egress from data plane.

Stack:

Layer	Tool	Purpose	Deployment
Metrics	Prometheus	System health, latency, throughput, resource utilization	VM on same cluster, 15s scrape interval
Dashboards	Grafana	Real-time dashboards, alerting visualizations	Same VM as Prometheus
Logs	Loki + Promtail	Structured logging, log aggregation	Promtail sidecars on each service
ML Model Registry	MLflow	Model versioning, training history, artifact store	On NFS, metadata in PostgreSQL
Tracing	Jaeger (optional)	Distributed tracing for agent churn model vLLM calls	Optional, ~5% trace sampling
Database	PostgreSQL	Interaction log, eval results, audit trail	Primary + replica, WAL backups

5.2 Key Metrics to Instrument

System Health Metrics

Metric	Thresholds	Trigger
vLLM queue depth	Alert if > 10	Model is backlogged; concurrent load too high
vLLM VRAM utilization	Alert if > 90%	May OOM on next large batch
vLLM P95 time-to-first-token	Alert if > 2.0s	Model is throttled; check GPU temperature
Agent response latency P95	Alert if > 5.0s	SLA breach; investigate tool latencies
Churn model latency P95	Alert if > 0.5s	Serialization / feature engineering bottleneck

Metric	Thresholds	Trigger
GPU memory utilization	Alert if > 85%	Approaching OOM; scale to GPU 3 if needed
GPU temperature	Alert if > 75C	Thermal throttling; check cooling

Query examples (Prometheus):

```
# vLLM queue depth
histogram_quantile(0.95, vllm_request_queue_depth)

# Agent response latency
histogram_quantile(0.95, http_request_duration_seconds{endpoint="/agent/query"})

# GPU utilization
nvidia_gpu_utilization{gpu="0"}
```

ML Quality Metrics

Metric	Check Frequency	Threshold	Action
Churn model AUC (held-out set)	Weekly batch job	Alert if < 0.78 (3% drop from 0.80)	Investigate data drift; plan retraining
Feature distribution drift (tenure, charges, score)	Daily aggregation	Alert if mean shifts > 2 from training baseline	Investigate data quality; label for retraining
Eval suite nightly run	Every night at 2 AM	Alert if any metric drops > 10% from previous night	Investigate; check if recent code changes broke eval
Eval judge consensus agreement	Per-run	Alert if any score differs > 1.5 between passes	Judge prompt may need recalibration

Example dashboard panels:

- "Churn Model AUC Over Time" (line chart, weekly points, baseline band)
- "Feature Distributions" (violin plot of recent vs. training)
- "Eval Suite Scorecard" (heatmap: test_case metric, red if <80%)

Agent Behavior Metrics

Metric	Signals
Tool call distribution	Histogram of which tools are called most. Alert if distribution shifts (e.g., lookup_customer rate drops agent not gathering context)
Tool error rate	% of tool calls that return error. Alert if > 5% (broken tools or bad data)
Escalation rate	% of sessions that escalate to supervisor. Alert if drops suddenly (model quality degradation hidden?) or spikes (too conservative)
Session length distribution	Median turns per session. Alert if mean turns > 5 (model looping or confused)
Response length	Avg tokens in final response. Alert if changes >20% (prompt or model behavior shift)

5.3 Alerting Rules

Three alert tiers:

Tier	Latency	Notification	Example
Info	530 min	Recorded in Prometheus, visible in Grafana	"GPU temperature 72C, up from 68C"
Warning	15 min	Email to team Slack channel	"vLLM queue depth > 10, might degrade SLA"
Critical	Immediate	Page on-call engineer	"Agent response latency P95 > 5s for 5+ min" OR "Churn model AUC dropped to 0.75"

Alert examples:

```
# Critical: SLA breach
- alert: AgentLatencyBreach
  expr: histogram_quantile(0.95, http_request_duration_seconds{endpoint="/agent/query"}) > 5.0
  for: 5m
  annotations:
    summary: "Agent P95 latency > 5s for 5 minutes"
    action: "Check vLLM queue, GPU temp, churn model latency"

# Warning: Model drift detected
- alert: ModelAUCDrift
  expr: churn_model_auc < 0.78
  for: 1h
  annotations:
    summary: "Churn model AUC dropped to {{ $value }}, baseline 0.80"
    action: "Review recent data; plan retraining"
```

6. Rollout and Rollback Strategy

6.1 Phased Deployment (8 weeks)

Phase	Week	Deliverable	Acceptance Criteria	Owners
0 Infrastructure	1	GPU drivers, network, NFS	All 4 GPUs recognized; NFS mounts < 10ms latency; DNS resolving	Infra/SRE
1 Churn Model Service	2	FastAPI service, model artifact (joblib)	AUC 0.80 on holdout; P95 latency < 200ms; can serve 10 req/s	ML Eng
2 vLLM Hosting	3	vLLM server + Mistral 7B INT4	P95 TTFT < 1.5s; tool calling works on 10 test prompts	ML Eng + Platform
3 Agent Integration	4	Agent service, end-to-end testing	Eval suite passes: tool accuracy 80%, judge avg 3.5/5, P95 < 5s	Agent Eng
4 Observability	5	Prometheus, Grafana, Loki, MLflow, Postgres	All dashboards populated; 3 test alerts fire and route correctly	Platform + SRE
5 Load Testing	67	Simulate 10 concurrent reps, sustained 1 hour	P95 latency stays < 5s; no OOM; eval runs nightly without impact	QA + Platform
6 Cut over	8	Deploy to production, monitor	All acceptance criteria from phase 05 passed; zero egress events	All

6.2 Rollback Design

Principle: Each phase is independently reversible.

Service Rollback

Models, LLM, and agent service: Deploy behind version flags or on versioned endpoints.

```
# Deployment
/services/churn-model/v1 churn_model.joblib (v1)
/services/churn-model/v2 churn_model.joblib (v2)
/services/agent/v1 agent:v1 (GPT-4o era)
/services/agent/v2 agent:v2 (vLLM era, uses /services/llm/mistral)
```

```
# Rollback: DNS or service mesh routes to /v1 instead of /v2
# No code redeployment needed; zero downtime
```

Database Rollback

Interaction log is append-only; eval results are immutable (versioned).

```
-- Rollback interaction log: filter by timestamp, archive post-cutover interactions
SELECT * FROM interactions WHERE created_at < '2026-10-15T00:00:00Z' -- Keep
SELECT * FROM interactions WHERE created_at >= '2026-10-15T00:00:00Z' -- Archive to backup

-- Eval results: never overwrite; create new version
eval_results_v1_2026_09_15.json
eval_results_v2_2026_10_15.json ( Can delete if rollback)
```

Model Artifact Rollback

Models stored in versioned directories with SHA-256 checksums:

```
/models/churn/v1/model.joblib
/models/churn/v1/model.joblib.sha256
/models/churn/v2/model.joblib
/models/churn/v2/model.joblib.sha256

# Rollback: Service reads from /models/churn/v1 instead of /v2
# Checksum verification ensures no corruption
```

Deployment Script Idempotence

All deployment scripts are idempotent: running them 2 on v2 is safe.

```
# Example: deploy_agent_service.sh
if service_exists agent-v2; then
    echo "v2 already deployed, skipping creation"
else
    docker build -t agent:v2 .
    docker run -d --name agent-v2 agent:v2
fi

# Re-running this script is safe; it's a no-op if v2 exists
```

6.3 Acceptance Criteria for Production Promotion

Must pass **all** of the following before promoting Phase 6 cutover:

- [] Churn model AUC 0.80 on production-representative holdout set (verified 2 weeks before cutover)
- [] Agent P95 end-to-end response time < 5s under simulated 10-concurrent-user load (sustained 1 hour)
- [] Eval suite: tool accuracy 80%, LLM judge average 3.5/5.0 across all 5 dimensions (nightly for 2 weeks)
- [] Zero external network egress events in 72-hour pre-production test window (verified by network monitoring)
- [] Interaction log persistence verified: kill service, restart, confirm logs survive (test + verify in staging)
- [] Observability stack: all dashboards loading with real data, at least 3 test alerts fire and route correctly
- [] Rollback procedure tested end-to-end: promote v2 rollback to v1 promote v2 again (no data loss)

7. Risks and Open Questions

7.1 Top 5 Risks

#	Risk	Like liho od	Impact	Mitigation
R-1	LLM quality too low for reliable tool calling (Mistral 7B vs GPT-4o gap)	High	High	Prompt engineering + output validation + human review flag (90 days). Evaluate Llama 3.1 8B as Plan B. Fine-tune on domain data if resources permit.
R-2	Latency > 5s under real concurrent load	Medium	High	Load test vLLM aggressively (10+ concurrent, 1 hour). Pre-provision GPU 3 as capacity reserve. Cache frequent lookups. Consider batching if latency still high.
R-3	GPU hardware procurement delay (T4 lead time)	Medium	High	Procure immediately on plan approval. Parallel develop in cloud until hardware arrives. Budget contingency for alternative GPU (e.g., A100 rental for Q4 if T4 unavailable).
R-4	Eval judge score discontinuity (Mistral judge GPT-4o judge)	High	Medium	Re-baseline all scores using local judge before migration. Never compare old (GPT-4o) vs new (Mistral) judge scores. Document score shift in release notes.
R-5	Hidden external dependencies discovered late	Low	High	Dependency audit before Phase 1: network trace + grep for external URLs. Test in fully isolated network sandbox (unplug from internet). Run eval suite 100x; catch any "call home" behavior.

7.2 Open Questions (Answer Before Committing)

- **What is the expected peak concurrent user load in this market?**
 - Determines whether 2 T4 for vLLM is sufficient or whether GPU 3 must be added for redundancy
 - Example: If 50+ concurrent reps recommend 3 T4 for vLLM + failover
- **Is fine-tuning the local LLM on TeleConnect domain data permissible and resourced?**
 - Fine-tuning on ~500 labeled agent interactions would close the quality gap significantly (GPT-4o parity in 6090 days)
 - Requires labeled data pipeline, training infrastructure, compliance review

- Recommend yes if quality gap (Mitigation 4 in section 3.4) is insufficient by Month 3
- **What are the specific compliance requirements for interaction log retention and access control?**
- Affects database schema (PII encryption, field-level access control)
- Affects audit trail (who accessed what data, when)
- Recommend: 7-year retention, SHA-256 integrity hashing, role-based access (reps see own sessions, supervisors see all, compliance team read-only)
- **Is there an existing data pipeline for collecting new labeled data in this market?**
- Determines churn model retraining cadence and ownership
- If manual labeling required monthly cadence with 500-label batches
- If automatic labels (e.g., actual churn after 30 days) continuous, faster retraining
- **Can the local LLM be updated during a maintenance window, or does it require full deployment approval?**
- Affects how quickly quality regressions can be addressed
- Recommend: maintenance windows on Sundays 24 AM UTC (off-peak), 1-hour window for model swap
- If approval required each time longer time-to-fix for quality issues

8. Monitoring and Success Metrics

8.1 Launch Checklist (Week 8)

- ☐ All hardware online and validated (GPUs, RAM, storage)
- ☐ All services deployed and health-checked
- ☐ Eval suite passes all acceptance criteria for 2 consecutive runs
- ☐ Observability stack live with 7 days of baseline metrics
- ☐ Runbooks written: incident response, escalation procedures
- ☐ Team trained on monitoring dashboards and alert response
- ☐ Data migration complete (if migrating from cloud)
- ☐ Security audit passed (egress check, access control)

8.2 Success Metrics (Month 1 Post-Launch)

Metric	Target	Verify By
Agent response latency P95	< 5s	Prometheus dashboard, daily check
Churn model AUC	0.78	Weekly batch job on fresh data
Tool accuracy (eval suite)	80%	Nightly eval run
Escalation rate	515%	Interaction log analysis
System uptime	99.5%	Prometheus uptime alert

Metric	Target	Verify By
Zero data egress	100%	Network monitoring, egress alert
Interaction log durability	100%	Restart test weekly